

Data Collection and Preprocessing Phase

Date	19 June 2025
Team ID	SWTXT_20251747173488
Project Title	Productivity Prediction Web App
Maximum Marks	6 Marks

Data Exploration and Preprocessing: Productivity Prediction Web App

Saurabh Yadav

19 June 2025

Introduction

The Productivity Prediction Web App uses machine learning to forecast garment worker productivity, leveraging the "Productivity Prediction of Garment Employees" dataset. This document details the data exploration and preprocessing steps applied to ensure data quality and reliability for training Linear Regression ($R^2 = 0.197$), Random Forest ($R^2 = 0.547$), and XGBoost ($R^2 = 0.482$) models. Preprocessing addresses missing values, categorical encoding, and feature engineering to optimize model performance.

Data Source

- Dataset: "Productivity Prediction of Garment Employees"
- Source: Kaggle (<https://www.kaggle.com/datasets/utkarshs/productivity-prediction-of-garment-employees>)
- Format: CSV file (`garments\_worker\_productivity.csv`)
- Size: 1197 records, 15 features
- Features:
 - quarter: Quarter of the year (e.g., Quarter1, categorical)
 - department: Department type (sweing, finishing, categorical)
 - day: Day of the week (e.g., Monday, categorical)

- team: Team number (1–12, numerical)
- targeted_productivity: Target productivity (0.35–0.8, decimal)
- smv: Standard Minute Value (2.9–50.48, decimal)
- wip: Work in Progress (0–1591, integer, nullable)
- over_time: Overtime in minutes (0–25920, integer)
- incentive: Incentive amount (0–98, integer)
- idle_time: Idle time in minutes (0–6.5, decimal)
- idle_men: Number of idle workers (0–40, integer)
- no_of_style_change: Style changes (0–2, integer)
- no_of_workers: Number of workers (2–59, decimal)
- date: Observation date (e.g., 1/1/2015, string)
- actual_productivity: Target variable (0.2337–0.7548, decimal)
- Purpose: Train ML models to predict `actual\_productivity` based on 14 input features (excluding `date`).

Data Quality Assessment

- Missing Values:
 - * Feature `wip` has 506 missing values (42.3% of records), primarily for `finishing` department where WIP is not tracked.
 - * No other features have missing data.
- Duplicates:
 - * No duplicate records found (verified by checking all columns).
- Inconsistencies:
 - * `department` has inconsistent spellings: `sweing` (should be `sewing`) and `finishing.`
 - * `date` is a string; requires conversion to derive `month` feature.
 - * Categorical features (`quarter`, `department`, `day`, `team`) need encoding for ML compatibility.
 - * Numerical features (e.g., `smv`, `over\_time`) have varying scales, but tree-based models (Random Forest, XGBoost) are scale-invariant.

- Outliers:

* Features like `over\_time` (max 25920) and `wip` (max 1591) have large ranges but are valid in context (e.g., high overtime during peak production).

* No extreme outliers requiring removal identified.

Preprocessing Steps

1. Handling Missing Values:

- Imputed missing `wip` values with the mean (for `sewing` department records) using Pandas.
- Rationale: Mean imputation preserves data distribution for tree-based models; `wip` is missing systematically for `finishing`.

2. Correcting Inconsistencies:

- Standardized `department`: Replaced `sweing` with `sewing` using Pandas string replacement.

3. Feature Engineering:

- Derived `month` from `date` (converted to datetime) to capture temporal trends.
- Dropped `date` as it's redundant after extracting `month`.

4. Categorical Encoding:

- Applied `LabelEncoder` (Scikit-learn) to categorical features: `quarter`, `department`, `day`, `team`.
- Example: `quarter` (Quarter1–Quarter5) mapped to 0–4; `department` (sewing, finishing) to 0–1.
- Rationale: Label encoding is suitable for tree-based models; avoids high dimensionality of one-hot encoding.

5. Feature Selection:

- Selected 14 features as inputs: `quarter`, `department`, `day`, `team`, `targeted\_productivity`, `smv`, `wip`, `over\_time`, `incentive`, `idle\_time`, `idle\_men`, `no\_of\_style\_change`, `no\_of\_workers`, `month`.
- Excluded `actual\_productivity` (target) and `date`.

6. Data Splitting:

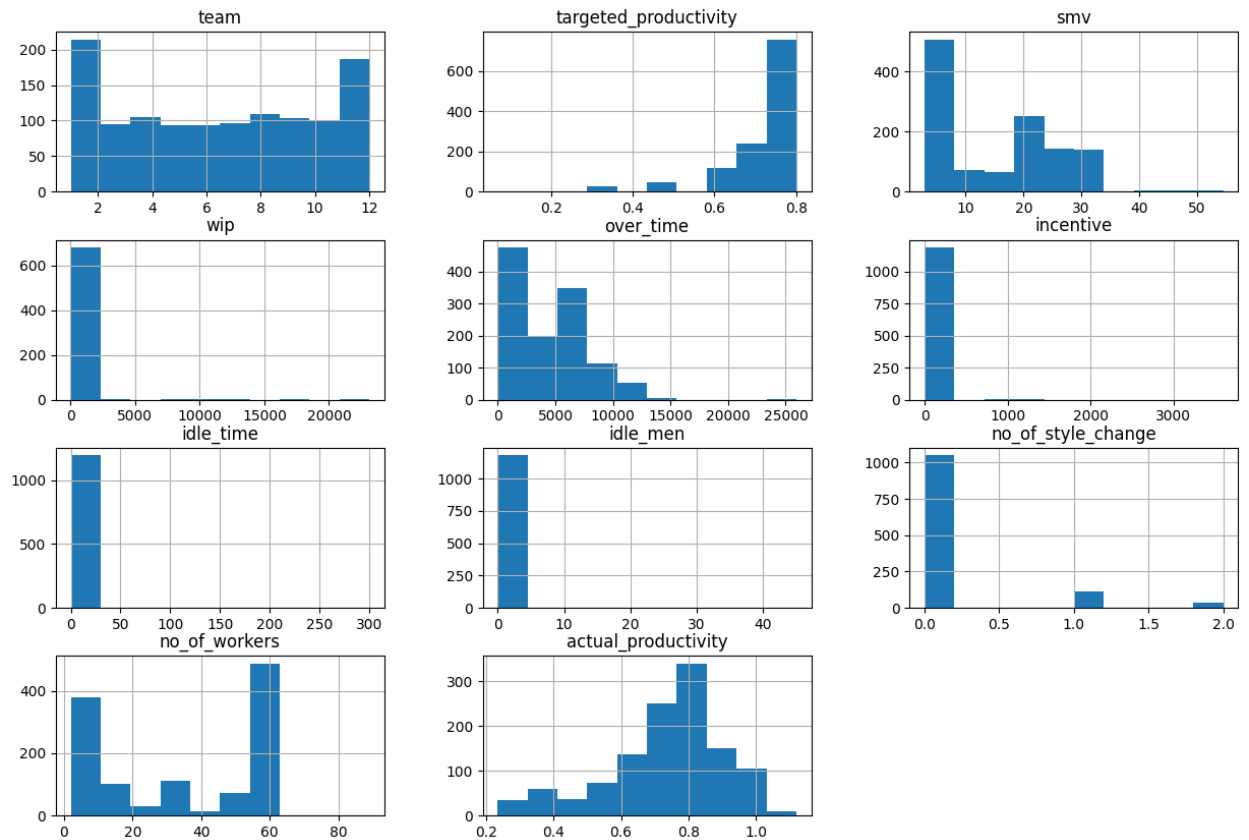
- Split dataset into 80% training and 20% testing sets (random_state=42) using Scikit-learn.
- Ensured consistent preprocessing for training and testing to prevent data leakage.

Impact on Analysis

- Preprocessing enabled robust model training, with Random Forest achieving the highest R^2 score (0.547), followed by XGBoost (0.482) and Linear Regression (0.197).
- Mean imputation for `wip` preserved data integrity, critical for `sewing` department predictions.
- Categorical encoding and feature engineering (e.g., `month`) captured key patterns, improving model performance over raw data.
- Standardized `department` names ensured consistency, avoiding model errors.
- The preprocessing pipeline, implemented in Python with Pandas and Scikit-learn, supports the app's real-time prediction functionality.

- Description: A heatmap visualizing missing values in the dataset, highlighting 'wip' with 42.3% missing data.
- Source: Generate using Seaborn (script provided below) or use a stock heatmap image from Unsplash (search 'data heatmap').

2. Figure 2: Feature Distribution Histogram



- Description: A histogram of the 'smv' feature, showing its distribution (range 2.9–50.48), critical for productivity prediction.

- Source: Generate using Matplotlib (script provided below) or use a stock histogram image from Unsplash (search 'data histogram').